Case Study Report

COMP6579001 - Big Data Processing

Implementation of Big Data Architecture for E-Commerce Analytics and Customer Satisfaction Prediction on Brazilian Olist Data

Group 3, Class LB01

Members :

2802462702	Christian Putra Dwiyan Setyo Pradana
2802543233	Dideo Setia Simangunsong
2802530590	Elizaveta Falisha Susanto
2802402395	Jason Alexander Wijaya
2802475320	Maurel Chairinniswah Yasmine
2802417560	Marcellino Wilson Rusli
2802402193	Maria Theresia, Sie
2802403126	Nabila Alya Mukhbita
2802542741	Ofa Sofwatan Munifa
2802539242	Steven Halbert Widjaja
2802403435	Timothy Jason Latief

Contents

Cover....1
Contents....2
Executive Summary....4
A. Introduction....5
1. Background....5
1.1 Business Domain....5
1.2 Problem Statement....5
2. Big Data Characteristics....6
2.1. Volume....6
2.2. Velocity....7
2.3. Variety....7
2.4. Veracity....7
2.5. Value....8
3. Business Motivation....8
4. Reasons for This Problem Considered as Big Data Issues....9
5. Limitations of Traditional Data Processing Approaches....10
B. Big Data Architecture Design....11
1. Architecture Overview....11
2. Data Sources....12
3. Ingestion Layer....12
4. Storage Layer....14
5. Processing & Analytics Engine....15
6. Visualization Layer....16
7. Monitoring & Security....16
8. Infrastructure Decision Analysis....17
C. Implementation Details....18
1. Data Processing....18
1. Data Load....18
2. Data Cleaning....18
3. Data Transformation....19
4. Agregation....21
5. Feature Engineering....22
2. Data Analytics....23
1. Analytics Objective & Problem Definition....23
2. Dataset Understanding for Analytics....23
3. Descriptive Analytics Implementation....24
4. Classification Model Development....25
5. Model Evaluation & Performance Analysis....26

6. Feature Importance & Business Insight Extraction.....27
3. Visualization..... 28
D. Result and Discussion.....30
1. Result & Business insights..... 30
2. Challenges & limitations.....30
3. Future Improvements.....32
a. Real-Time Streaming Integration.....32
b. Advanced Analytics Enhancement.....32
c. Distributed Infrastructure Scalability..... 33
E. Conclusion..... 33
F. Declaration Statement..... 34
Author Contribution..... 34
AI Assistance Declaration..... 36
Reference..... 37

Executive Summary

The E-commerce industry nowadays keeps growing and creates a big amount of transaction data that can be used to support decision making in business. The process of turning such a big amount of data to support decision making may present its own challenges during transaction analysis, building predictive models and assessing customer satisfaction. Hence, this project goal is building a big data architecture system that can process large amounts of data, making a machine learning model to predict overall customer satisfaction and also business insights.

The Brazilian E-Commerce Public Dataset by Olist is used in this project. The dataset contains various e-commerce transactions data in Brazil. This dataset includes a wide range of transaction data such as customers, payments, orders, products, delivery, geolocation, suppliers, and customer reviews. The dataset was selected because it has a sufficiently large and comprehensive amount of data for aggregation and is suitable for implementing big data technologies from a business analytics perspective.

The project started with an architectural framework using a distributed file system (HDFS) in a virtual machine environment. Apache Spark is also used as a working framework for processing data. The ETL process is performed to extract, clean, and transform the data before using it for machine learning and analysis. The resulting data is presented in an interactive dashboard that allows for easy management and analysis of business data.

In the machine learning and analytics stage, this project predicts customer satisfaction with classification methods in machine learning that use models such as Catboost and Random Forest with ADAYSN techniques and Gaussian noise augmentation as a comparison. Both models' training processes are evaluated using the 10-Fold Cross Validation method based on Stratified Shuffle Split, so the model is proven to have strong and stable discriminatory capabilities that are directly validated through key performance metrics such as accuracy, precision, recall, F1-Score and AUC-ROC. From the modeling results carried out, the catboost classification model provides the best performance in prediction with an accuracy of 81.98% and successfully maps the features that most determine whether a customer will be satisfied or not.

According to the analytics process, it was found that the main features that influence customer satisfaction are delivery time, delivery delay, logistics efficiency across regions, and order fulfillment complexity. Customer satisfaction drops significantly with long delivery times and large orders may have a higher risk of fulfillment issues. These results also indicate that delivery-related features have a significant impact on customer satisfaction, making logistics efficiency a key factor in maintaining customer satisfaction. The development of predictive models and business dashboard may help companies by supporting data driven decision making especially to improve customer experience.

A. Introduction

1. Background

1.1 Business Domain

E-commerce is a sector of businesses that offers the wholesale sale of goods through a digital platform or marketplace that brings buyers together with sellers. The e-commerce business sector possesses various entities such as sellers, payment systems, consumers, logistics, and review systems from consumers. There are various business activities in the e-commerce ecosystem such as ordering of products by customers, payment of transactions, as well as giving reviews by customers after the products are received. The activity generates massive amounts of data that include customer information, products, sellers, transactions, payments, as well as customer reviews. This data can be leveraged to understand consumer behavior, sales performance, item sales patterns, as well as the effectiveness of the delivery process. Large-scale application of data analytics can enable e-commerce companies to gain insights to improve user experience, increase efficiency in operational, as well as maximize sales strategies.

1.2 Problem Statement

All of the transactional data holds a lot of important information that can be leveraged to understand the business activity taking place on the platform. Improperly conducted analytics will make it difficult for companies to understand customer behavior patterns, product performance, as well as operational efficiencies. Those conditions may lead to potential insights from the available data not being optimally exploited.

Below are some insights that can be analyzed from the dataset :

- Identify best-selling products and categories to help companies determine inventory and promotional strategies
- Analysis of customer behavior such as purchase frequency and transaction value to understand customer characteristics
- Evaluate seller and logistics performance, especially related to delivery times and customer satisfaction levels
- Analyze customer reviews, to know the factors that influence ratings as well as user experience
- Analyze sales trends over time, to see patterns of transactions occurring over a given period

The utilization of big data technology allows the dataset analysis process to be carried out more effectively and may generate a range of relevant insights for

companies in understanding business performance more thoroughly. The insights gained can assist companies in formulating more appropriate strategies based on data. Improving the quality of service to customers can also be supported through the utilization of the results of the data analysis.

2. *Big Data Characteristics*

There are 5 characteristics that signify a data can be categorized as Big Data. These 5 characteristics are Volume, Velocity, Variety, Veracity, and Value. The Brazilian E-commerce dataset created by Olist is one of the real examples of Big Data as it fulfills the characteristics of the 5V of Big Data. This dataset has a huge volume, consisting of 100,000 orders in 9 tables with a total of millions of rows of data. Data diversity is also high as it contains Structured, Unstructured data, and semi-structured data.

2.1. Volume

Volume refers to the amount of data that must be stored and processed in a Big Data system. The Ollist Brazilian E-Commerce data set consists of several tables representing e-commerce activities such as orders, payments, products, customers, and reviews from customers.

Orders	±100.000 orders data
Order Items	±110.000 product item bought
Customers	±99.000 customers
Payments	±100.000 payment transaction
Reviews	±100.000 customer reviews
Products	±30.000 products

TABLE 1. Summary of Data Structure and Volume of the Main Dataset

2.2. Velocity

Velocity refers to the rate at which data is generated, enters the system, and is processed. In the Olist dataset, data comes from e-commerce activities that occur continuously and almost in real-time such as when customers order products, systems record transaction payments, logistics updates delivery status, and customers provide product reviews.

In an actual e-commerce system, this data can generate thousands of transactions per day and must be processed quickly for a variety of needs, such as monitoring daily sales, analyzing product performance, detecting delivery delays, and product recommendations.

2.3. Variety

Variety refers to the diversity of data types in big data. In big data there are 3 types of data, namely unstructured, semi-structured and structured data. Structured data is strictly formatted data, usually in the form of rows and columns with a specific data type such as csv or excel. Unstructured data is data that has no specific format, and is typically in the form of free text, images, video, or audio. Semi-structured data is a data format that falls somewhere between structured and unstructured, the common format is usually in the form of JSON, XML, or a table with columns containing long text.

3 types of data types in the Olist dataset:

- An example of structured data can be found in table olist_orders_dataset. This table contains structured order data and each column has a clear data type and a specific purpose.
- An example of unstructured data can be seen in table olist_order_review_dataset. In one of its reviews, this column contains free texts written by customers in Portuguese about their shopping experiences.
- An example of semi-structured data can be seen in table olist_geolocation_dataset, this table has clear columns, however its relationships with other data are not always rigid. One zip code can have multiple different coordinates because data is collected from multiple sources. This creates a semi-structured relationship where 1 zip code entity is connected to multiple values that may not always be consistent.

2.4. Veracity

Veracity refers to the level of trustworthiness, accuracy, and cleanliness of data. In the Olist dataset, an example is :

- Incomplete Data
  In the presence of important columns such as reviews (review_comment_message) whose contents are missing, this could lead to analysis bias.

- Data Inconsistency
  There was a data oddity in the booking status section, there was an order canceled but having review data, logically it couldn't happen. The possible date format is different across various tables, so it needs to be standardized.
- Data Duplicate
  In the geolocation table itself there are 200,000+ duplicates, because a single zip code has different latitude and longitude coordinates, although normals still have to be handled with care.
- Unreasonable Data
  From the analysis came some rather illogical data such as very high payments, for example more than 12000, and very long delivery times that reached 9 days.

2.5. Value

Value refers to the benefits that can be extracted from data to support business decisions. Value potential looks like :

- Logistics Optimization
  With data available employee performance can be measured, for example courier performance from location data, delivery time, and geolocation.
- Quality Improvement
  Reviews can be a source of identification for problematic products, and causes of dissatisfaction.
- Sales Prediction
  Data history can be used to predict demand, plan stocks, and detect seasonal patterns.
- Sales Segmentation
  Customers can be grouped based on frequency, transaction value, or category preferences. Data can be used for loyalty programs, recommendations, and identification of long-term customers.
- Seller Performance Analysis
  Evaluate sellers based on shipping speed, product accuracy, and reviews. Data can be used to provide coaching and teamwork evaluation.

3. Business Motivation

Big data analysis on the Olist Brazilian E-Commerce dataset was conducted to help companies understand the business activities occurring in the e-commerce platform. The stored transaction data includes various important information such as product orders, payments, shipping as well as customer reviews that can be utilized to acquire business.

Through the utilization of big data technology, companies can identify customer buying patterns, know the most desired products, as well as evaluate seller performance and delivery processes. The information can be used to improve sales strategies, optimize operational, as well as improve customer satisfaction. So, a

collection of values on the data can have a meaningful meaning by bridging between the techniques in analyzing the data with the business needs so that the data has a very meaningful meaning. The impact of a company that has known the meaning in terms of business on the dataset is that the company can take appropriate decisions by thinking of strategies on how sales and purchasing activities can be improved so that the company gets more profit than before.

4. Reasons for This Problem Considered as Big Data Issues

E-commerce datasets are categorized as big data issues because they have characteristics such as enormous volumes of data, variations in data types, as well as data that is continuously increasing along with transaction activity on marketplace platforms. Some aspects of Big Data present in this dataset include:

1. Volume (Large Amount of Data)

This dataset includes several extensive tables, such as orders, order items, customers, products, sellers, payments, and reviews that contain hundreds of thousands to millions of transaction data. The sheer magnitude of this amount of data makes processing with conventional database systems less effective, thus requiring big data technology for distributed data storage and processing.

2. Variety (Data Variance)

This dataset does not consist of only one type of data, rather it contains various types of interconnected data such as unstructured, structured and semi-structured. from various tables like:

- Order transaction data
- Customer Data
- Seller data
- Product Data
- Payment data
- Customer review data

This diversity of data structures demonstrates the complexity of an e-commerce system involving multiple business entities within a single ecosystem.

3. Velocity (Rapid Data Growth)

In e-commerce platforms, transactions take place continuously on a daily basis. Impact, data regarding sales, payments, and customer reviews experienced rapid growth. Analytic systems must be capable of processing this constantly growing data to provide business insights immediately.

4. Value (Value of Information)

This abundant transaction data has significant value for companies to analyze consumer behavior, knowing the products of most interest, assessing salesperson performance, as well as improving the quality of service and delivery.

5. *Limitations of Traditional Data Processing Approaches*

In handling large-scale data, traditional systems such as single RDBMS often face serious hindrances. These conventional approaches have limitations in handling modern data characteristics which include volume, velocity, and variety. Some of the major points that are the limitations of traditional systems are:

1. Vertical Scalability Issues

Traditional systems typically rely on vertical scaling, where performance is increased by adding hardware specifications (such as RAM or CPU) on just one central server. The problem is, this method has a maximum physical limit and the cost is very expensive. In contrast to big data ecosystems that use horizontal scaling, where we can add many standard computers (commodity hardware) to a single cluster to distribute the workload.

2. Rigid Data Schema

Relational databases rely heavily on persistent schemas. That is, data must be validated and "forced" into a predefined table format before it can be saved. This becomes a major constraint if we have to process unstructured data such as activity logs, data from social media, or documents whose format is indeterminate and changes.

3. Limited Processing Speed

Traditional data processing is sequential or sequential in nature. Once the data volume goes into the terabyte or petabyte scale, the query process becomes very slow. Big data infrastructure addresses this with parallel and distributed processing, where data is fragmented to be worked on simultaneously by many machines at once through available nodes.

4. Infrastructure Cost Burden

Managing giant data in conventional enterprise databases takes a huge investment, both for software licensing and the procurement of expensive dedicated servers. The big data approach is much more cost-efficient due to its open-source nature and is designed to run optimally on standard hardware without sacrificing overall system performance.

B. Big Data Architecture Design

1. Architecture Overview

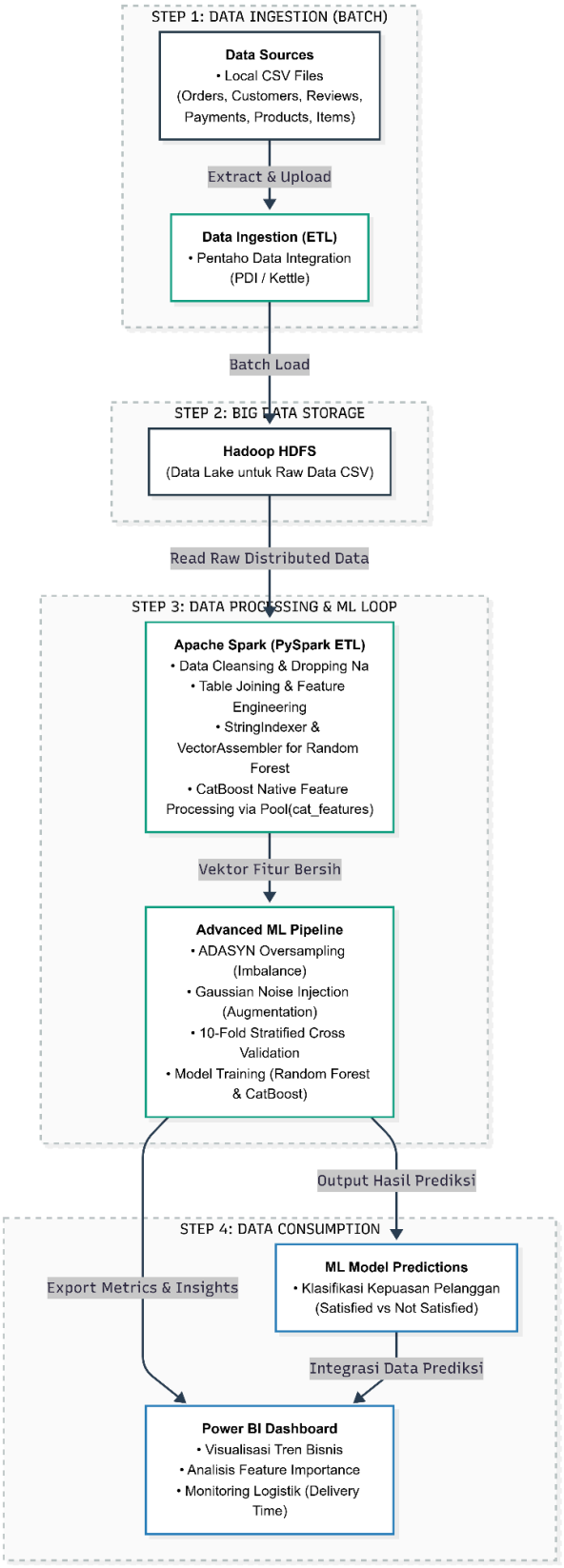


Figure 1. Big Data Architecture for Customer Experience Analytics

This architecture is designed to process and analyze the Brazilian Ollist e-commerce dataset that has 5V characteristics (Volume, Velocity, Variety, Veracity, and Value). The chosen architecture is a layered approach (layered architecture) oriented to batch processing, as the data used is historical in nature and does not demand analysis in real-time. This approach ensures scalability and efficiency in handling large volumes of data.

This architecture consists of several main layers that work sequentially to transform raw data into business insight:

- Data Sources: Raw data of e-commerce transactions in CSV format.
- Ingestion Layer: Performs the process of loading data from source to storage using the Ingestion Batch ETL pattern.
- Storage Layer: Responsible for storing distributed data using HDFS as primary storage for large volumes.
- Processing & Analytics Engine: Uses Apache Spark as the primary processing engine to run transformations, analytics, and machine learning (customer satisfaction classification) models.
- Visualization Layer: Convert analytics results into easy-to-understand visualizations, such as interactive dashboards, using tools like Power BI.

The choice of the overall batch architecture is justified because key business analyses (such as customer segmentation and performance evaluation) can be performed periodically, which is more efficient and optimal for large-scale data already collected.

2. *Data Sources*

Data source is the retrieval of data from a specific source in the form of raw data for further processing as in this case, the data is retrieved in CSV format. Although the CSV format already belongs to the structured data groups, it is still not tabular in shape but it is in the form of a comma-separated simple text file format. Still, the format is easy to process because of its simpler structure than other formats. Therefore, the format is an appropriate choice for the data source as it is supported by database applications, spreadsheets, and other data tools.

3. *Ingestion Layer*

Data ingestion is a process in big data that involves loading data from one or multiple sources to a storage that may be in the cloud or not in order to be further processed for analysis and modeling [1]. Ingestion methods that can be used are batch, streaming (real-time) or a combination of both (hybrid).

Batch Ingestion is a process in which data is collected in large batches and delivered at a specific time (e.g. daily or weekly) [2]. This method is suitable when there is no need for real-time updates, data is historical in nature and large volumes must be moved effectively without latency pressure. Streaming Ingestion means processing

data continuously as data is generated or entered [2]. This method is suitable for real-time analytics such as fraud detection or product recommendations that must respond quickly to user behavior.

Based on the Brazilian Online Shop List dataset which has characteristics in the form of periodically updated transaction data and product catalogs, then the most suitable method selected is batch processing because the data are generated in large quantities and does not require real-time processing. Therefore, the batch processing method was selected for ingestion because the used dataset has historical characteristics and its analysis is not necessary in real-time.

ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) are two commonly used ingestion patterns in big data. The main difference lies in the transformation process before or after the data is loaded into storage. ETL performs transformation first before loading to the warehouse so this pattern is suitable for data that needs to be cleaned and standardized first for analysis, while ELT performs transformation after loading to storage [3].

Based on the datasets used, the ETL pattern is selected because the transaction data and its details need to be cleaned and standardized before entering the data warehouse in order to ensure the consistency and quality of the data for further processes such as sales analysis and customer segmentation. Additionally, based on the characteristics of the Big Dataset's velocity and volume. thousands to millions of transactions, the incoming data speed is relatively stable because the data can be processed in batches instead of real-time. Additionally, data also has different types of data types so it needs standardization before further processing. Therefore, batch data collection using ETL meets the technical requirements for managing large quantities, stable speed, as well as complicated diversity.

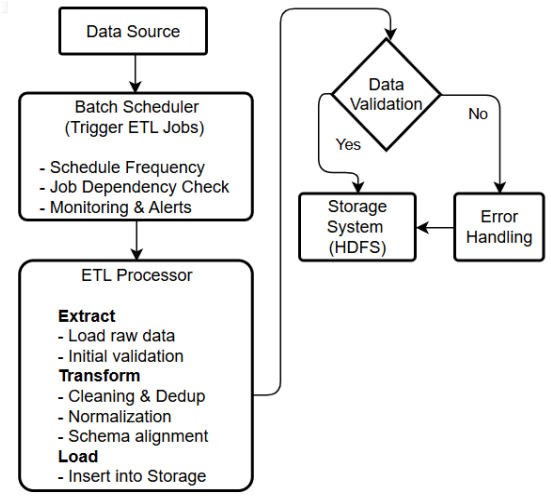


Figure 2. Data Ingestion and ETL Processing Workflow

The layer ingestion diagram illustrates the data flow high-level starting from source data (CSV) towards storage. The data is first run through a batch of schedulers whose

task is to trigger the ETL process periodically, checking for dependencies not otherwise as well as monitoring the running of the pipeline. Then the data goes into the ETL Processor where the extract stage takes the raw data, transforms performs the cleaning, deduplication, normalization and schema adjustment, while the load puts the data into storage. After that the data is validated to determine if the data is ready to go into storage or needs to go into the error handling mechanism if it is invalid.

In line with that flow, the ingestion process is supported by the use of tools that play a role in ensuring each stage runs efficiently and structured. The dataset used is in csv format and processed using Python with pandas library due to its capability in reading as well as manipulating structured data flexibly. Next, the data that has gone through the ETL process is stored in the form of structured tables on the database / data warehouse to support more complex analytics needs. The static nature of the dataset makes the frequency ingestion performed in batches with a one-time load approach, making it simpler and efficient without requiring repetitive processing mechanisms.

All these processes are concluded with a data validation process that serves to ensure that data quality is maintained by performing checks against the number of records, the appropriateness of the data structure, as well as the consistency of the values in order to avoid any anomalies that could affect the results of the analysis. This approach is supported by the latest research stating that the application of ETL in the ingestion layer is capable of improving the integrity, consistency and quality of the data before it is used in the process of big data analysis[4].

4. Storage Layer

The storage layer's task is organizing and storing the data that has gone through the ingestion process. Considering the characteristics of the Olist dataset and the needed analytical requirement, selecting the correct storage technology is crucial to ensure the system's effectiveness, growth, and reliability. From the dataset it can be seen that the state of the data is quite large and consists of many tables, is structured and semi structured in nature and not just real time data(used for analysis).

HDFS works by breaking data files into small blocks (typically 128 MB per block) then those blocks are spread across multiple servers in the cluster. Each block is copied multiple times (replication) for security. In this project HDFS is chosen as primary storage because:

- Large volume of data : the dataset consists of 9 tables with a total of millions of rows of data. HDFS is specifically designed to store and process large-scale data efficiently.

- Suitable for batch processing : since the analysis is performed in batches (not necessarily real-time), HDFS supports the processing of large amounts of data simultaneously with high speed (high throughput).
- Horizontal scalability : HDFS can add regular servers (hardware commodity) into the cluster, cheaper and more flexible compared to traditional systems that rely on just one big server.
- Failure-resistant : HDFS automatically duplicates (replicates) data to multiple servers. If one server breaks down, data remains safe.

Therefore, HDFS is the best choice as the primary storage in this project because the dataset is large in size and used for analysis, HDFS is designed to store and process big data efficiently, and supports analytic data workflow.

5. Processing & Analytics Engine

In the Big Data architecture, the processing & analytics engine is the core component responsible for running manipulation, transformation, and analysis on data at a large scale. In contrast to traditional systems that rely on sequential processing on a single server, the engine is designed to distribute the workload load across a cluster of machines. Its main goal is to extract value from the raw data that has been stored in the storage layer.

The processing & analytics engine has the responsibility of performing manipulation, transformation, and analysis on the transformed large-sized data, so as to generate business insights (business insights). In this architecture, the processing engine and the analytics engine are divided into two main focus areas:

1. Processing Engine (Data Processing)

Apache Spark: The main data processing in this pipeline is done by Apache Spark. Apache Spark was selected because of the superiority of its performance level in distributed processing of big data.

Batch Processing Approach: Given the nature of the Brazilian E-Commerce dataset which is historical and static in nature, data processing does not require the presence of real-time processing. Therefore, the data were processed through a batch processing approach by Apache Spark before being stored and visualized.

Python & Pandas: During the processing stage which requires flexible reading and manipulation of formatted CSV data, processing is done with the help of Python programming language and Pandas library.

2. Analytics & Machine Learning Engine (Advanced Analytics)

Customer Satisfaction Classification Model: As an attempt to gain deeper business insights, this machine learning model predicts the classification level of customer satisfaction level (Customer Satisfaction).

Target Definition: The satisfaction classification is defined into two classes based on review_score. Review scores ≥ 4 were classified as satisfied customers (Satisfied or value 1), while review scores < 4 were classified as dissatisfied customers (Not Satisfied or value 0).

6. Visualization Layer

The final stage of the Big Data implementation process is the visualization phase, which transforms the generated data into visual forms such as charts, dashboards, and reports. Its goal is to support data-driven decision-making based on an understanding of operational performance and linking it to actual business needs [5].

Parquet is used to optimize the data, removing irrelevant data and keeping only the most relevant data. The study then examines the distribution of sales and whether sales are seasonally dependent.

The Brazilian e-commerce dataset is static, not dynamic, so real-time processing is unnecessary; the project instead uses a batch visualization technique. After preprocessing in Apache Spark, the data is saved and visualized periodically. Batch processing is simply the most sensible option for a historical dataset like this; setting up a full streaming infrastructure would be unnecessarily complicated and offer no real benefit.

7. Monitoring & Security

Monitoring in big data systems aims to monitor the passage of the data pipeline thoroughly, starting from the process of ingestion to processing. In architectures that are distributed in nature, failures can occur at multiple points, thus requiring a monitoring mechanism capable of detecting problems early on. On this project, monitoring was focused on a few key components. On the ingestion layer that uses the ETL hatch approach, monitoring is required to ensure every process is running properly, from data retrieval to loading to storage. Failures in this process, such as errors during transformation or incomplete data, can directly impact the quality of the analysis. Therefore, the system needs to be equipped with activity recording(logging) as well as notification in case of process failure.

On the storage layer that uses HDFS, monitoring is performed to maintain the stability of the storage system. These things monitored include node condition, storage capacity as well as data replication mechanisms. Because HDFS works distributedly, it is important to ensure that data remains available even if a disruption occurs on one of the nodes in the cluster. Monitoring also helps detect possible

bottlenecks or performance degradation that can affect the data processing process, to support those processes, tools like Apache Ambari or Grafana can be used to help visually and real time performance of the cluster[7].

In addition to monitoring, the security aspect also plays an important role in big data architecture because the processed data includes customer transaction information as well as e-commerce activities. Security is required to ensure that the confidentiality, integrity and availability of data is maintained during the storage and processing processes take place. In the Hadoop ecosystem, one of the commonly used security methods is Kerberos-based authentication to restrict cluster access to only authorized users [8].

Monitoring and Security support each other to maintain the reliability of the big data system. Abnormal activity, such as repeated login attempts, cluster performance degradation, as well as illegal access can be detected through monitoring tools and recorded in system logs for auditing as well as troubleshooting needs. Big data systems are not only capable of managing large-scale data efficiently, but also capable of maintaining the safety, quality and stability of the data processing process, with the appropriate application of monitoring and security.

8. *Infrastructure Decision Analysis*

The infrastructure in this project was designed based on the characteristics of the Brazilian Olist E-Commerce dataset which contains large scale historical transaction data with high volume and variety. A distributed big data architecture was implemented to improve the efficiency of processing analytics and machine learning workloads compared to traditional single-node systems.

The project used a virtual machine environment since it provides easier configuration and lower deployment costs during the setup of Hadoop HDFS and Apache Spark. HDFS was used as the main storage system since it is capable of storing distributed data across multiple nodes and handling large datasets more consistently. Since the project mainly focused on historical transaction records, batch processing was considered sufficient without the need to implement real-time streaming.

The project also relied on Apache Spark to process the data in parallel, especially during transformation, feature preparation and machine learning stages. In the preprocessing stage, Python and Pandas were also utilized to clean and prepare CSV data before further processing in Spark.

Power BI was chosen to visualize the final results because it provides interactive dashboards that are easier to understand during analysis. Through these visualizations, the processed data and prediction results could be presented more clearly. Overall the infrastructure used in this project was able to run the data processing workflow and customer satisfaction prediction process properly.

C. Implementation Details

1. *Data Processing*

Data processing is an important phase in this project. It works to transform raw data into a readable format that is usually designed for machine learning models or other advanced algorithms. Based on the pipeline, this process is done in 5 stages:

1. *Data Load*

The first step is to load the raw dataset into the HDFS ecosystem. The data loading process is performed using Apache Spark (PySpark) after initiating a SparkSession session. The raw dataset is imported from the Storage Layer i.e HDFS, located inside a Cloudera virtual machine environtment in path hdfs://localhost:8020/user/cloudera/olist/raw. There are total 9 raw dataset files that loaded into the Spark DataFrame, which is:

- olist_orders_dataset
- olist_customers_dataset
- olist_order_items_dataset
- olist_order_reviews_dataset
- olist_order_payments_dataset
- olist_products_dataset
- olist_geolocation_dataset
- olist_sellers_dataset
- product_category_name_translation

To optimize the process of reading data structures, the spark.read.csv function is written together with the header=True and inferSchema=True parameters for all 9 dataset files. The header=True parameter is used so that the first row of the dataset is recognized as the name column. The inferSchema=True parameter is used so that Spark can detect and automatically convert data types automatically, for example, by mapping time formats to timestamp data types or numbers to integer or double data types.

The process of reading the olist_order_reviews_dataset.csv file apparently requires extra handling because the review_comment_message column is filled with messy comment text, ranging from special characters to newlines. To prevent the DataFrame structure from breaking or shifting during import, additional arguments such as quote='"', escape='"', and multiLine=True were intentionally added to the read function.

2. *Data Cleaning*

After the raw dataset is successfully loaded into Spark, the process continues to data cleaning. This step is important because data quality affects the performance of the model. For a very large and multidimensional dataset like

Olist, missing values and other anomalies need to be fixed so it won't affect the relationship between the tables.

3 main steps on data cleaning:

Our cleaning process uses PySpark with three main steps:

- Order Status Filtering
  Since the focus of the analysis is on delivery metrics and customer satisfaction, we only need the completed transactions. We use .filter() function to keep the order where order status == 'delivered', which will remove other types of order status like canceled or pending, because it is not relevant with delivery time calculation.

- Handling Missing Values
  Handling of null values is done with two approaches:
  - Dropping Rows
    Missing values can break table join so we use .dropna() to remove any rows with missing values, such as customer_id, order_id, and order_delivered_customer_date.
  - Imputation
    Some of the important rows also have missing values, so we fill them with an empty strings or a template string such as "No Review" using .fillna().

- Duplicate Data
  There are also some duplicates in optional columns like review_comment_message or review_comment_title which doesn't need to be dropped, many customers skipped these part. We handle these parts with .fillna(), replacing them with "No Review".

- Outlier Handling using IQR Method

  Some financial feature like price or freight_value have spikes in the value which can make the model to be not accurate. We use IQR to fix these problems.

  - Use the percentile function to find Q1 (25%) and Q3 (75%).
  - Use conditional filters to drop any transaction rows that exceed the standard 1.5 * IQR limit.

3. Data Transformation

After the dataset has been cleaned up from duplicate rows, missing value, and outlier. The next step is Data Transformation. This step focused on things like reshape rows structure, correcting data type, and reformatting columns so that it can fit to a model. Because of the usage of PySpark, data transformation cannot be executed directly. Instead the transformation will utilize lazy evaluation,

which means that Catalyst Optimizer handles and simplifies all of the query for an optimal performance work.

There are 3 main points that is done in Data Transformation stage, which is:

- Timestamp Casting

  When importing the Olist dataset, PySpark automatically reads date and times data as a standar string text. Because it is unable to count the delivery speed feature from a raw string, the conversion of the temporal column into timestamp date type is mandatory to do. With combining the .withColumn() column with PySpark date handling feature, the date column inside the Data Frame is being converted. All of the converted date columns are:

  - order_purchase_timestamp
  - order_approved_at
  - order_delivered_carrier_date
  - order_delivered_customer_date
  - order_estimated_delivery_date

  This conversion process ensures that the ISO standard time format (YYYY-MM-DD HH:MM:SS) is mapped correctly, so that date-time functions can be executed without triggering calculation errors.

- Standardization of Categorical and Text Variables

  It is often that a raw dataset is in a mess, including the Olist dataset. It is often to meet an inconsistent capitalization and space that is not needed in a word. If ignoring this problem, Spark will treat this identical term with a small difference like capital and non-capital letter or additional space as a different entity. This difference will affect how the data will be grouped. To fix this, standarization for consumer data text with combining a couple of string functions from PySpark is needed. This standarization is done in:

  - lower() function
    Using lower() function to make every text inside the categorical column like order_status and product_category_name into a non-capital letter. This ensure that the difference between capital and non-capital letter did not ruin the grouping

  - Trim() function
    Using the trim() function for the next step cleaning. This function is used to remove additional space that is unintentionally left by customer or seller during data input.

This standarization part is an important stage before joining the main tabel with the seach table *i.e* product_category_name_translation. Because the Olist category is written in Portuguese, any spelling incompability or typo will ruin the mapping and translating to inggris. This step is to ensure that every key relation is fully accurate during the join process

- Numeric Casting

  To reduce excessive memory usage in the HDFS cluster and improve the efficiency of the machine learning workflow, several schema adjustments are applied. Columns containing financial values such as payment_value, product price, and freight_value are converted into the double data type by using the .cast("double") function. Meanwhile, fields that represent count-based values, including payment_installments, are changed into integer format with .cast("int"). This adjustment helps allocate memory more efficiently during the data processing stage.

4. Agregation

Before tables can be joined, features must first be aggregated. This step is important because the Olist database schema has several many-to-one relationships connected to the main olist_orders_dataset table. If tables are joined directly without preprocessing, the number of rows can increase uncontrollably due to the combination of duplicate records. Therefore, to avoid this problem, PySpark functions such as .groupBy("order_id") and .agg() are used so that each table only stores one data representation for each order_id before the merge process is performed.

The aggregation process on three DataFrames:

- Order Item table (df_order_items)

  A single order can contain multiple products with varying quantities and item types. Therefore, data is aggregated at the order level by counting the number of item records in each order using the count() function. The result is stored as the total_items feature, which is later used to support Insight 4 analysis. Additionally, the sum() function is used to calculate the total product cost (total_price) and accumulated shipping costs (total_freight) for each order.

- Payment table(df_order_payments)

  In the df_order_payments table, some customers use more than one payment method for a single order, for example, using both a voucher and a credit card. If this data were combined directly, the data rows could be duplicated because a single order has multiple payment records. To prevent this, the data is aggregated by order number (order_id), then all payment

amounts are added together. This way, we get a single total payment value (total_payment_value) for each order.

- Review table (df_order_reviews)

  When examining the df_order_reviews table, we discovered that a single order can receive more than one review. To avoid duplicate data, the review scores were first averaged so that each order (order_id) had only one final rating. This average score was then converted into two simpler rating options, such as satisfied or dissatisfied. This way, we can more clearly see the level of customer satisfaction with each order.

5. Feature Engineering

After the data was cleaned and combined, the next step was to generate additional information from the original Olist transaction data. The goal was to transform the raw data tables into more useful markers or information to aid in the process of grouping the data into two categories. All of the information was designed to align with the five business analysis objectives identified at the outset of the project.

Feature engineering implemented includes:

- Classification Label (is_satisfied)

  The is_satisfied value is generated based on the number in the review_score column and will serve as the main target for future predictions. To simplify this process, customer reviews are divided into two groups. Scores 4 and 5 are converted to a 1, indicating a satisfied customer. Conversely, scores below 4 are assigned a 0, indicating a customer is less or not satisfied with their order.

- Delivery Duration (delivery_time)

  Delivery duration (delivery_time) is calculated to determine how long it will take for an order to reach the customer. This calculation is done by finding the difference in days between the order date (order_purchase_timestamp) and the time the item is received (order_delivered_customer_date). The final result will show the total number of days the item has been in transit, from the time the customer completes the order until the item arrives at its destination.

- Lateness Indicator (is_late)

  A special marker is created to determine whether an order is late (is_late). This is done by comparing the actual delivery time to the original estimated delivery date (order_estimated_delivery_date). If the order arrives later than the estimated delivery date, it will be given a score of 1. Conversely, if

the package arrives on time or even earlier than estimated, the order will be given a score of 0.

As a final step in data preparation, all the separate datasets are combined into a single central table called df_master. This central table holds all previously generated calculations and new information. This consolidation process uses the df_orders_clean table as the primary reference, and then adds various additional information drawn from customer data, reviews, payments, and order details.

To avoid problems in subsequent processing, rows without customer satisfaction information (is_satisfied) will be removed. Furthermore, any remaining blanks after the tables are merged will be filled with 0s. Once the data is completely clean and organized, the dataset is then transferred to the storage layer (HDFS). The data is stored in a special format called Parquet to maintain its original structure and safely prepare it for subsequent analysis.

2. *Data Analytics*

1. *Analytics Objective & Problem Definition*

In the e-commerce industry, retaining customers is much more efficient than acquiring new ones. The main problem faced is that there is a level of customer dissatisfaction that is not always detected directly through formal complaints. By predicting satisfaction based on transaction and delivery behavior , companies can conduct proactive interventions before customers switch to competitors.

The main goal is to build a predictive model capable of identifying whether a customer will feel satisfied (satisfied) or dissatisfied (not satisfied) towards the order based on operational data. The classification approach was chosen because the target variable we wanted to predict was categorical in nature (binary), i.e. separating customers into two groups: “satisfied”(1) and “dissatisfied”(0). These algorithms allow businesses to get clear labels for decision-making.

Customer Satisfaction Prediction Goal:

- Project Target : achieve a classification model with high performance that is capable of handling data imbalance (class imbalance).
- Analytic goal : discover dominant factors affecting satisfaction and provide an early system for operational management teams to improve services

2. *Dataset Understanding for Analytics*

The dataset used in this study is an amalgamation of several e -commerce tables such as orders, customers, payments. Once the merging process is

conducted, the data will be processed and cleaned to address inconsistent data as well as remove attributes that are irrelevant to the modeling process. The final result of preprocessing generated a total of 95,830 transaction data as well as 11 major features used in the classification process. Some features used include 'delivery_time' to represent the order delivery duration, 'is_late' for delivery delay indicator, 'total_payment_value' and 'total_price' describe the transaction value of the customer, as well as , 'total_apitems quantity per product 'show.

The target variable used is 'is_satisfied', which is a label that represents the level of customer satisfaction based on the results of the data distribution analysis. In this section the class imbalance (class imbalance) on the target variable is found. The satisfied customer class dominates the dataset with a total of about 75,701% transactions (79%) while dissatisfied customers account for about 20,129 transactions (21%). Such a condition becomes one of the challenges in the modeling process, as it can create a model that tends to predict the majority class. Therefore, the study applies a class weighting approach that aims to make the performance of the model remain balanced in recognizing both categories of customers.

3. *Descriptive Analytics Implementation*

Before the customer satisfaction classification (is_satisfied) modeling step, descriptive analysis was performed on Olist Brazilian E-Commerce dataset to comprehend the fundamental features of the data. Summary statistics showed that there was a substantial class imbalance in the target variable, with most consumers falling into the satisfied category and a tiny minority into the dissatisfied category. Initial data showed significant differences in logistics performance between late status (is_late) and delivery time (delivery_time), which are supposedly the main causes of complaints. Because these properties were found early on, the team included automatic class balancing settings, such as auto_class_weights="Balanced," in the algorithm during the modeling process.

In addition to logistical features, the first correlation research included operational and financial aspects such as total products per transaction (total_items), customer region grouping (customer_state_idx), and total payment value (total_payment_value). Using the data summary function, it was discovered that the distribution of payment values had a wide range, requiring the use of a specialized technique, such as a log transform, to stabilize the data variance prior to entering the algorithm. Since descriptive observations of these operational patterns provide a strong statistical foundation showing delivery performance and order complexity are strongly

related to satisfaction indicators, all attributes entering the training phase have significant commercial value.

4. *Classification Model Development*

```
cb_model = CatBoostClassifier(
    iterations=2000,
    learning_rate=0.03,
    depth=8,
    eval_metric='AUC',
    auto_class_weights='Balanced', # Changed from scale_pos_weight to auto_class_weights
    l2_leaf_reg=5,
    random_seed=42,
    task_type='CPU',
    early_stopping_rounds=100,
    verbose=100
)

cb_model.fit(
    train_pool,
    eval_set=val_pool
)
```

Figure 3. Snippet of CatBost Model using Auto Balancing Feature

The development of this classification model was carried out using PySpark MLlib and the catboost algorithm as the main model and the random forest classifier as the comparator algorithm. The CatBoost model was chosen as the primary model because it has the built-in feature of Ordered Boosting that can prevent overfitting and has a built-in encoding target that is capable of converting categorical data to numbers automatically without corrupting the data pattern.

```
# — StringIndexer untuk customer_state
state_indexer = StringIndexer(
    inputCol="customer_state",
    outputCol="customer_state_idx",
    handleInvalid="keep"
)
```

```
# — VectorAssembler ————————
assembler = VectorAssembler(
    inputCols=FEATURE_COLS,
    outputCol="features"
)
```

Figure 4. Snippet of Feature Engineering in Random Forest Model

The process is composed in a machine learning flow that includes pre-processing steps such as StringIndexer to convert geographical data (customer_state) into numbers, and VectorAssembler to integrate various features such as delivery time, delay status, and transaction values of the random feature vector model forest. Both models were evaluated by a 10-fold cross validation method in which the model will perform training and testing as many as 10 times to ensure the performance results of the model are stable in nature, objective, and unbiased on one partition of the data only. The purpose of these augmentations is to ensure the consistency of data transformation during the training and testing phase.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

Figure 5. Snippet of Train Test Split Method

The model training phase started with splitting the cleaned and aggregated data using a train test split technique. We allocate 80% of the data for model training and 20% of the data for the testing phase. We also use a random seed of 42 to ensure repeatability. Since our data has a class imbalance in which the satisfied customers are 79% while the unsatisfied customers are only 21%, the model uses a proportional class weighting technique to make sure the model pays more attention to the unsatisfied class. Minority categories are given greater weight so the model won't lean towards the majority class only but also pays attention to the minority class. By using that technique, detecting potential customer dissatisfaction can be more accurate. However, the models have a difficulty setting a boundary between two classes due to the data overlapping condition between classes so the model accuracy suffers from stagnation.

5. *Model Evaluation & Performance Analysis*

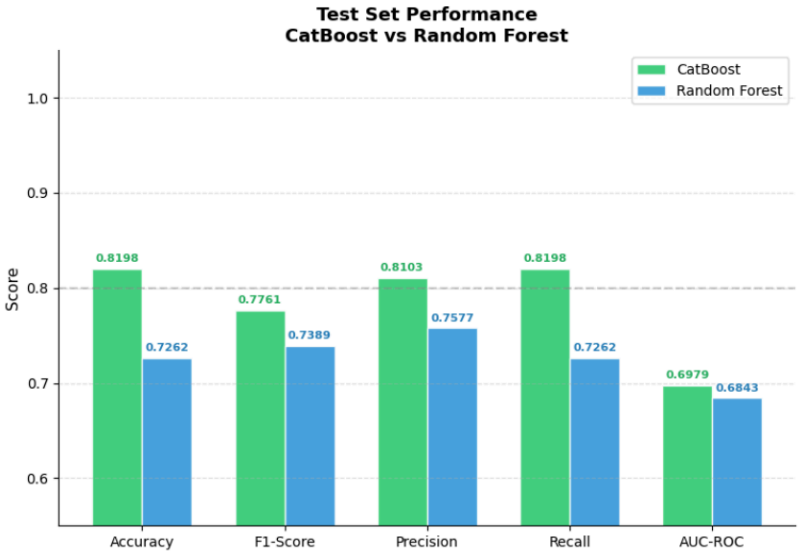


Figure 6. Comparison on Evaluation Metrics in Both Model

Both of the predictive models (catboost and random forest) are evaluated by using accuracy, F1-Score, precision, recall, and AUC-ROC metrics. However

since the data is imbalanced and very overlap, the F1-Score is used as the primary metric since it is capable of balancing the precision and providing similar recall for both classes. Additionally, confusion metrics are used to show how well the model performs in predicting the actual label, and the AUC-ROC metric ensures the ability of the model to effectively distinguish between satisfied and dissatisfied customers across various classification thresholds.

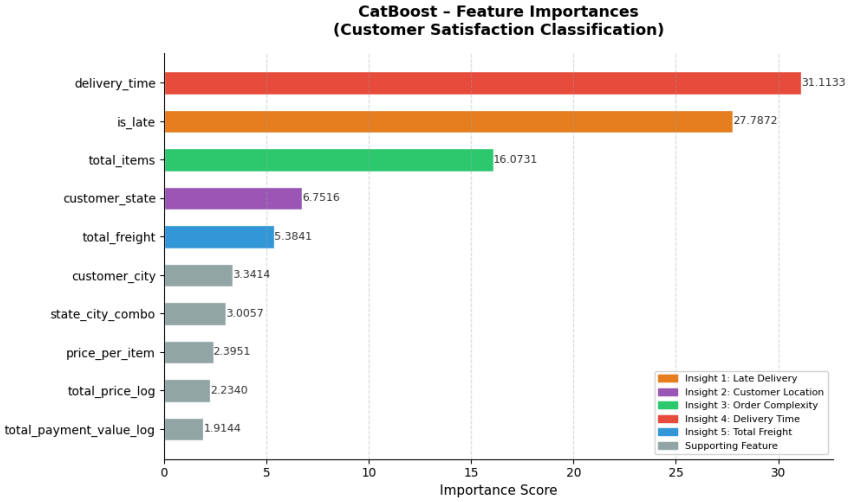


Figure 7. The Feature Importance Analysis Result of Catboost Model

From the feature importance analysis it is evident that the attribute 'delivery_time' with a value of 31,11 and 'is_late' with a value of 27,79 are the top 2 factors contributing in the prediction of customer satisfaction. This shows that the reliability of the delivery system and the delivery schedule have a greater influence than price or payment method. The findings provide a basis for business planning to focus on improving operational efficiency and accuracy of delivery time estimates to reduce customer dissatisfaction in the future.

6. *Feature Importance & Business Insight Extraction*

Logistics indicators continue to be the most important factors in determining customer happiness, according to feature importance analysis taken from the enhanced CatBoost Classifier model. The most significant predictors, according to the model weighting findings, are the delay (is_late) and delivery_time variables, which are followed by operational and geographic factors like total_items and customer_state_idx. From the business analysis, these insights demonstrate that on-time delivery has become an essential element in e-commerce. The presence of the delay indicator (is_late = 1) has a very strong negative effect and immediately turns the model's prediction to customer dissatisfaction, regardless of the transaction amount.

The two factors that have the greatest impact on the risk of order fulfillment are geographic variability (customer_state_idx) and order size (total_items). Because the regional infrastructure in Brazil is heterogeneous, making it difficult to maintain consistent delivery windows from one state to another, shipments with many items often cause operational bottlenecks. In monetary terms, total_payment_value has a fairly limited overall impact, but it was retained in the model after logarithmic transformation because it still represents an important outcome: expensive customers often have different expectations about the delivery performance. Overall, these results suggest that the most effective ways to reduce transit delays are to improve supply chain operations and create more accurate geographic maps.

3. Visualization

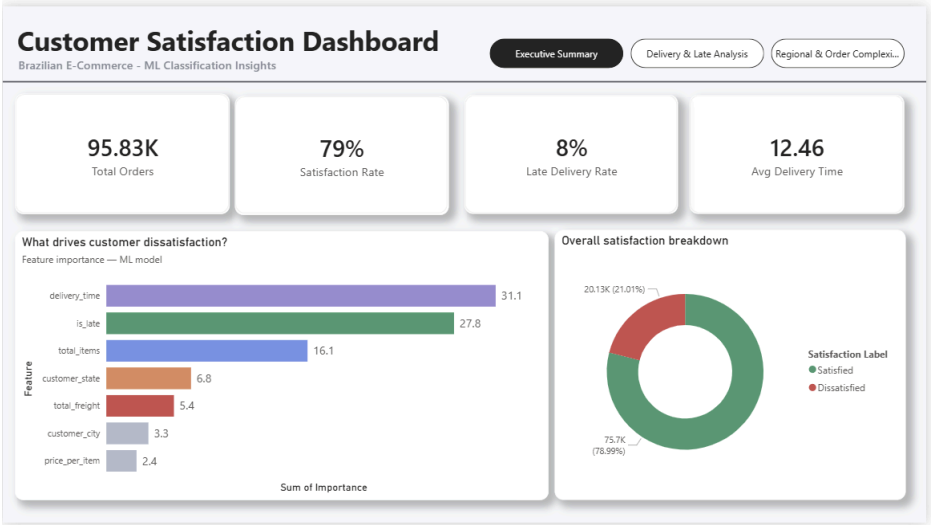


Figure 8. Executive Summary Analysis Dashboard

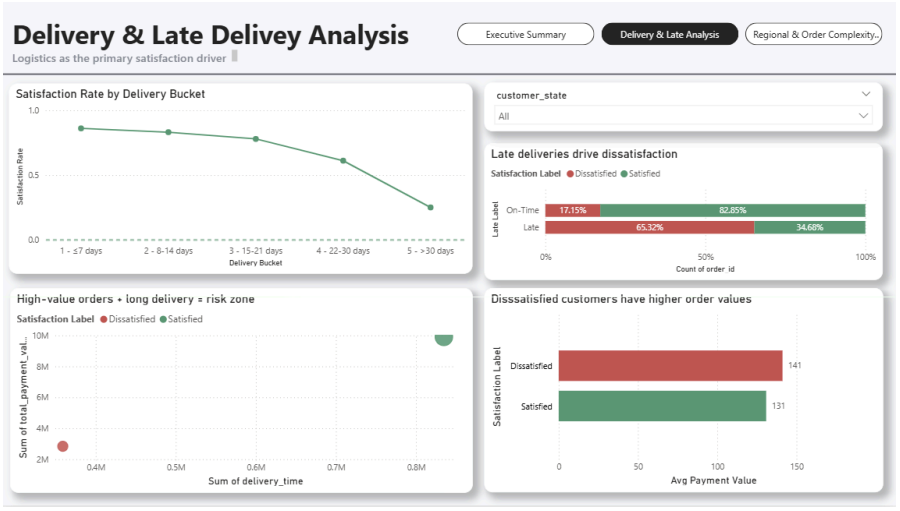


Figure 9. Delivery and Late Delivery Analysis Dashboard

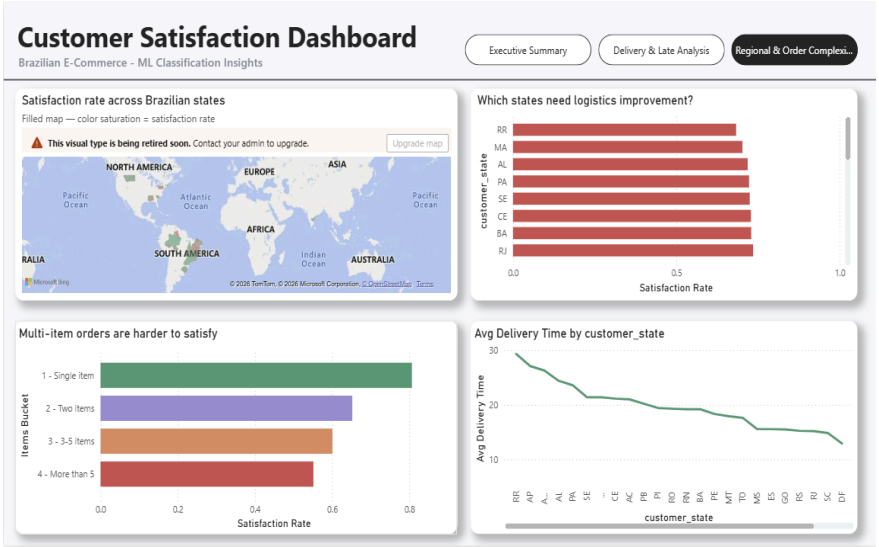


Figure 10. Regional and Order Complexity Analysis Dashboard

The visualization implementation phase was carried out by an interactive dashboard developer to facilitate the analysis of logistics performance and customer satisfaction through informative visualizations. The dashboard includes KPI cards to display key business information such as average daily time, late delivery rate, satisfaction level, and total orders. These KPIs are provided to help users understand the operational conditions of the business more quickly. The exclusive summary page also features a donut chart visualization to show the proportion of satisfied and dissatisfied customers. Additionally, a donut chart was chosen to visualize the overall customer experience during the ordering process. Figure 8, the exclusive summary dashboard, also features a bar chart to illustrate important features and their impact on customer satisfaction.

Next, the delivery and late delivery analysis dashboard is shown in Figure 9. This dashboard features a stacked bar chart to visualize the comparison of customer satisfaction levels between orders that arrive on time and those that arrive late. A line chart also supports in-depth analysis to identify changes in customer satisfaction trends based on delivery duration. A scatter plot is also available to identify patterns of dissatisfaction risk in specific transactions.

The final dashboard, Figure 10, features a regional and order complexity analysis dashboard designed to support analysis of differences in customer satisfaction levels between states. This map visualization uses a red color is indicating low customer satisfaction and a green color indicating high customer satisfaction. To aid analysis, a bar chart compares the average delivery state between states. Furthermore, an item bucket chart was created to identify differences in satisfaction levels depending on the number of items ordered. Overall, the creation of this three-page dashboard aims to help this organizations explore data

more efficiently so that they can generate business insights that can support future business operational decisions.

D. Result and Discussion

1. *Result & Business insights*

The analysis shows that delivery performance plays an important role in customer satisfaction. Delivery time and shipping delays were the factors that affected customer ratings the most. Customers usually gave better ratings when their orders arrived on time, with satisfaction reaching around 82.85%. However, when deliveries were delayed, the satisfaction rate dropped to around 34.68%, and more customers expressed dissatisfaction.

The same trend can also be seen from delivery duration. Orders delivered in less than 7 days had satisfaction rates close to 90%, while orders that took more than 30 days only reached around 33%. This indicates that longer waiting times can reduce customer satisfaction and negatively affect the overall shopping experience.

Differences between regions were also visible in the analysis results. States such as RR, MA, AL, PA, and SE showed lower satisfaction rates, and these areas also had average delivery times of around 20 to 30 days. Besides delivery speed, the number of items in an order also seemed to influence satisfaction levels. Orders with only one item achieved satisfaction rates of around 81%, while orders containing more than five items only reached about 55%. This could happen because larger orders usually involve more complicated packing and shipping processes, which increase the possibility of delays or delivery issues.

From these findings, it can be seen that improving logistics and fulfillment processes is important for maintaining customer satisfaction. Several improvements can be considered, such as adding fulfillment centers in regions with lower satisfaction rates, improving cooperation with local courier services, and placing popular products closer to customers to reduce delivery time. Better handling for multi-item orders may also help reduce shipping problems, for example through additional checking before shipment and more organized delivery management. These improvements can help create a better customer experience and support customer retention in the long term.

2. *Challenges & limitations*

a. Data Coverage and Referential Integrity Issues

There are still several missing values in the Olist dataset. These missing values are not ordinary missing values because they can affect referential integrity between tables. Order transaction records between customers and sellers in

the olist_sellers table have a zip_code_prefix that is not available in the olist_geolocation_dataset table. This invalid zip code between these 2 tables can causes some rows of data to be unable to connect with location information, thus making geographic analysis less complete. Furthermore, the store names in customer comments were intentionally disguised as Game of Thrones house names from the raw dataset, likely to protect the privacy of the stores mentioned in the review text. To address this, a special regex tool was used to remove false labels before entering the text-based modeling or NLP process.

b. High Numbers of Financial Outliers

Transaction values in the financial table, the olist_order_payments table, show significantly more outliers than expected. Nearly 8 out of every 100 entries in the payment_value column fall outside the normal range. The price column also has a similar number of outliers. However, the freight_value column is much more extreme, with more than 10% of the total data falling outside the outlier limits. Due to the large number of outliers, removing them is impossible, as it would impact both model performance and the original values, which were initially large. Therefore, a capping method was used, changing the values in the payment_value, price, and freight_value columns that exceeded the outlier limits to match the outliers themselves. This method ensures that data is maintained without disrupting relationships between tables, and maintains consistency between them.

c. Class Imbalance and Low Data Separability

Most of the Olist records show that nearly eight out of ten customers are satisfied. That is why this has become a real challenge. Even when adjusting the system's focus on dissatisfied customers, the results remain virtually unchanged. The purchasing behavior of satisfied and dissatisfied customers appears too similar in the log records. These patterns overlap, preventing the score from ever exceeding 81.98%. That upper limit remains firmly unchanged.

d. Infrastructure and Batch Processing Constraints

One part runs on a single-node Hadoop system via Cloudera, while most of the work is done in Google Colab. Everything is processed in stages, so there is no choice but to wait until all the sales data has been fully processed before accessing the Olist records. While this method effectively handles data cleaning and loading, it prevents the system from capturing user changes in real time. Therefore, when it comes to quickly detecting suspicious activity and responding to changes immediately, this system is inherently inadequate by design.

3. *Future Improvements*

a. Real-Time Streaming Integration

Further development on this project can be directed toward implementing a real time streaming architecture to address the limitation of the batch processing approach currently being used. On the current implementation, transaction, payment and delivery data are processed after historical data is collected so the insights generated cannot yet be used for live monitoring. By integrating Apache Kafka as a distributed messaging system, e-commerce activity data such as delivery sites, payments and customer activity can be processed continuously as the data enters the system. This approach allows companies to detect potential delivery delays or declines in service quality earlier before impacting customer satisfaction.

Additionally, Apache Spark Structured Streaming can be utilized to perform distributed and near real time processing of streaming data. Regarding this project, the technology can support the monitoring of delivery delays, changing customer transaction patterns as well as logistics performance more dynamically compared to the traditional batch approach, with the implementation of real time data processing, the analytical dashboard not only displays the latest historical data but also informs the company more accurately. making operational decisions more responsively and efficiently.

b. Advanced Analytics Enhancement

Several classification algorithms such as XGBoost, Random Forest, and CatBoost have been tested using various approaches such as Gaussian Noise, class weighting, ADASYN oversampling, ratio boosting, log transform and cross validation. The evaluation results show that most of the models still experience stagnant performance and the CatBoost model so far gives the best results with accuracy reaching 81.98% but its performance can still be further developed.

For future development, it can try some other models suitable for handling class imbalance and data overlap such as SVM (RBF Kernel), TabPFN and Neural Network to compare the performance of the model against the dataset. Further development can focus on the addition of more representative features such as features describing customer behavior as well as customer segmentation. Additionally a re-experiment can also be conducted against the target labeling strategy by diverting the aggregation method to assert the decision boundary causing the low separability problem on the current model.

c. Distributed Infrastructure Scalability

Future work will concentrate on switching from a single-node environment to a Multi-Node Hadoop Cluster architecture in order to improve the system's capacity to manage the exponential rise of e-commerce data volumes. Data storage via HDFS (Hadoop Distributed File System) can be duplicated across several worker nodes by putting in place a distributed cluster, which automatically increases data availability and fault tolerance. Because the resource manager will dynamically reroute data processing activities to other available nodes within the cluster, this enterprise-grade scalability enables the system to be stable even in the event of a hardware component failure.

Through in-memory parallel processing, the integration of Distributed Spark Processing on this multi-node architecture would greatly speed up machine learning modeling execution times. Future significant computational loads, such the CatBoost algorithm's decision tree iterations and the numerous evaluation procedures utilizing 10-Fold Cross Validation, which demand a lot of processing power, will depend heavily on this distributed design. In addition to being prepared to handle massive volumes of historical data, the infrastructure developed using this enterprise-level scalability strategy offers a strong basis for effective advanced analytical experiments.

E. Conclusion

Based on the results of the Big Data Analysis project using the Brazilian Public E-Commerce dataset, it can be concluded that big data technology can help the process of processing large amounts of e-commerce data in a more effective way. The deployment of Hadoop Distributed File System (HDFS) and Apache Spark in a virtual machine environment successfully supports distributed storage, processing, and transformation of data. Additionally, the ETL process performed also helps generate data that is more ready for analysis and machine learning.

At the analytic stage, the created machine learning model was successfully used to predict customer satisfaction based on transaction and delivery data with an accuracy of 81.98%. From the importance feature analysis, it is evident that factors such as delivery time, delay, quantity of goods purchased, and delivery cost have a significant impact on customer satisfaction. These things show that logistics performance and quality of delivery services are crucial aspects of maintaining customer experience on e-commerce platforms.

On this project is found an interactive dashboard developed to help the process of monitoring the customer satisfaction and logistics performance of the business more effectively. Additionally, the dashboard aims to facilitate the identification of patterns of delay, delivery, delivery duration, logistics performance between regions and identify order fulfillment complexities that affect customer satisfaction levels. Based on the results of the conducted analysis, it was found that the main problem of the

business lies neither in pricing nor payment but rather in the quality of supply chain and logistics execution. Therefore, this project proves that a combination of analytical dashboards and machine learning models can support companies to make data-driven decisions so that companies can improve service quality, improve logistics performance and create a better customer experience sustainably.

F. Declaration Statement

Author Contribution

Name of Author	BP	BDJ	AD	INF	DP	SP	ML	VI	DOC	PC
Maria Theresia, Sie	✔		✔				✔		✔	✔
Christian Putra Dwiyan	✔	✔			✔	✔			✔	✔
Dideo Setia Simangunsong			✔		✔	✔			✔	
Elizaveta Falisha Susanto	✔	✔					✔		✔	✔
Jason Alexander Wijaya	✔		✔					✔	✔	✔
Marcellino Wilson Rusli	✔		✔					✔	✔	✔
Maurel Chairinniswah Yasmine	✔			✔				✔	✔	✔
Nabila Alya Mukhbita	✔		✔					✔	✔	✔
Ofa Sofwatan Munifa	✔			✔			✔		✔	✔

Steven Halbert Widjaja			✔		✔	✔			✔	
Timothy Jason Latief			✔		✔	✔			✔	

Notes:

BP ⟶ Business Problem Definition

BDJ ⟶ Big Data Justification

AD ⟶ Architecture Design

INF ⟶ Infrastructure Decision Analysis

DP ⟶ Data Processing

SP ⟶ Spark Processing Implementation

ML ⟶ Machine Learning / Advanced Analytics

VI ⟶ Visualization & Insight Generation

DOC ⟶ Documentation & Report Writing

PC ⟶ Project Coordination

AI Assistance Declaration

During the project working process Big Data Processing aids AI technology as a supporting tool in multiple stages of project development. The use of AI is undertaken to assist to provide reference in the development of pre-processing program code, data modeling and dashboard visualization.

At the data preprocessing stage, AI is used as a supporting tool to help provide syntax references related to the data cleaning stage i.e. the creation of outlier detection functions on the numerical columns of the dataset as well as their handling when using capping methods. Additionally at the feature engineering and aggregation stage, AI is also used to reference code and help when errors occur in syntax and written code. AI is also used as a syntax reference when performing integrity referential checking between tables to check if the relationship of each primary key of the table is interconnected and valid. Lastly, Ai is also used in helping to export processed datasets from the Spark Notebook system into the HDFS ecosystem in order to produce the actual final dataset, then final exporting of the dataset from the HDFS ecosystem into the Cloudera VM system

At the modeling stage, AI is used as a supporting tool to help provide references regarding classification algorithm selection, class imbalance handling strategies as well as model performance evaluation. In this project AI is leveraged to help provide examples of code implementations using PySpark, MLlib, such as feature engineering processes, modeling pipelines, class weighting, cross validation, as well as to evaluate metrics such as accuracy, precision, recall, f1-score, AUC and ROC. AI helps provide suggestions for the experiments of the models used namely Random Forest, XGBoost, CatBoost which is useful for comparing the performance of the models implemented on the customer satisfaction prediction dataset

At the visualization dashboard creation stage AI is used to help provide additional column recommendations that support the visualization, visualization design, as well as color theme selection and help explain incrementally how to use PowerBI when a new feature is discovered that the team hasn’t used before. Then, dashboards are developed and readjusted by the team in order to represent business information more accurately, interactively and easily understood.

Although AI was used as a supporting tool during the project working process, the entire process of evaluation, result validation, data interpretation, as well as the final content of the report was still performed and double-checked by the team independently to ensure that the results obtained remained accurate, relevant, and in line with the project objectives.

Reference

[1]Šprem, Š., Tomažin, N., Matečić, J., & Horvat, M. (2024). Building Advanced Web Applications Using Data Ingestion and Data Processing Tools. Electronics, 13(4), 709. https://doi.org/10.3390/electronics13040709

[2] Mavrogiorgou, A., Kiourtis, A., Manias, G., Symvoulidis, C., & Kyriazis, D. (2023). Batch and Streaming Data Ingestion towards Creating Holistic Health Records. Emerging Science Journal, 7(2), 339–353. https://doi.org/10.28991/ESJ-2023-07-02-03

[3] W. Lutfe, "Data Integration Models in Enterprise Systems: A Comparative Study of ETL and ELT Methodologies A Comparative Study of ETL and ELT Methodologies," Jan. 2026, Available: https://www.researchgate.net/publication/400076663_Data_Integration_Models_in_Enterprise_Systems_A_Comparative_Study_of_ETL_and_ELT_Methodologies_A_Comparative_Study_of_ETL_and_ELT_Methodologies

[4] Jasti, N. V. K., & Ravi, V. (2025). *Enhancing data integration and quality using ETL processes in big data systems*. International Journal of Emerging Research in Engineering and Technology. https://ijeret.org/index.php/ijeret/article/view/474

[5] Kirti Mahajan, L. A. Gokhale (2017). *Significance of Digital Data Visualization Tools in Big Data Analysis for Business Decisions*. International Journal of Computer Applications.

[6] Joseph Kobi (2024). *Developing Dashboard Analytics and Visualization Tools for Effective Performance Management and Continuous Process Improvement*. International Journal of Innovative Science and Research Technology.

[7] I. Fedorovych et al., "Performance Benchmarking of Continuous Processing and Apache Spark Structured Streaming," CEUR Workshop Proceedings, 2024.

[8] P. Shen, X. Ding, and W. Ren, "Research on Kerberos Technology Based on Hadoop Cluster Security," in *Proc. 2nd Int. Conf. Advances in Energy, Environment and Chemical Science (AEECS 2018)*, 2018, pp. 195–198, doi: 10.2991/aeecs-18.2018.40.